DEPI · GRADUATION PROJECT

Enterprise DevSecOps E-Commerce Platform

Engineering Design & Implementation Handbook

Flask Microservices | Docker & Kubernetes | Terraform & AWS | CI/CD Security | Prometheus & Grafana

Document Version: 1.0

Classification: Internal — Development Team

Audience: Backend, Cloud, DevOps, DevSecOps, QA Engineers

Enterprise DevSecOps E-Commerce Platform

Engineering Design & Implementation Handbook

Graduation Project — DEPI (Digital Egypt Pioneers Initiative) Version: 1.0 **Classification:** Internal — Development Team

Table of Contents

---

1. Executive Summary

1.1 Project Goal

The **Enterprise DevSecOps E-Commerce Platform** is a graduation project that simulates a real-world, production-grade software delivery lifecycle. Rather than building "just another e-commerce app," the goal is to demonstrate mastery of the full chain of modern software engineering discipline: microservice design, containerization, automated security-integrated CI/CD, Infrastructure as Code, Kubernetes orchestration, and observability — all running on AWS.

The backend is composed of five Flask microservices (Product, Cart, Inventory, Order, Notification) that share a single PostgreSQL database and a Redis cache layer. The existing frontend is consumed as-is; this project does not redesign it.

1.2 Why This Architecture Was Selected

Decision	Reasoning
Microservices over monolith	Demonstrates service decomposition, independent deployability, and team-scalable ownership — a core DevSecOps competency employers look for.
Shared PostgreSQL (not database-per-service)	Keeps the project's data layer manageable for a student team while still preserving service boundaries at the code and API level. This is a deliberate, documented trade-off (see 1.4).
Amazon EKS over ECS/Fargate	Kubernetes is the industry-standard orchestrator; EKS experience is the most transferable skill for DevOps/Cloud roles.
Terraform over ClickOps or CloudFormation	IaC is a non-negotiable DevSecOps practice. Terraform is cloud-agnostic and the most widely requested IaC tool in the job market.
GitHub Actions over Jenkins	Native integration with the source repository, zero infrastructure to maintain, and the fastest path to a working pipeline for a student team.
Security-in-the-pipeline (SAST, secret scanning, dependency scanning, image scanning)	Embeds the "Sec" in DevSecOps rather than bolting security on at the end.
Prometheus + Grafana + CloudWatch	Combines open-source, portable observability (Prometheus/Grafana) with native AWS visibility (CloudWatch) — a very common enterprise pattern.

1.3 Enterprise Design Decisions

- **Immutable infrastructure:** All infrastructure changes flow through Terraform; no manual console changes are permitted after Phase 5.
- **Immutable images:** Every deployment uses a uniquely tagged Docker image (git SHA), never `latest`, to guarantee reproducibility and easy rollback.
- **Shift-left security:** Static analysis, secret scanning, and dependency scanning run on every pull request — before code ever reaches `main`.
- **Everything as code:** Application code, infrastructure code, Kubernetes manifests, and pipeline definitions all live in Git and are peer-reviewed.
- **Least privilege:** IAM roles, Kubernetes service accounts, and security groups are scoped to the minimum required permissions.
- **Observability by default:** Every service exposes a `/health` endpoint and Prometheus-compatible metrics from day one, not retrofitted later.

1.4 Known Trade-offs (documented intentionally)

- A **shared database** simplifies transactions across services (e.g., Order Service reading Inventory data) at the cost of tighter coupling. In a full enterprise system this would be split per service with an event bus; this is explicitly out of scope to keep the project achievable in a semester.
- **Notification Service is mocked** (e.g., logs instead of sending real emails/SMS) to avoid third-party account dependencies and cost.

1.5 Expected Outcome

By the end of this project, the team will have:

- A working multi-service Flask backend running on Amazon EKS.
- A fully automated CI/CD pipeline enforcing quality and security gates.
- AWS infrastructure fully defined in Terraform, destroyable and re-creatable on demand.
- Centralized monitoring dashboards (Grafana + CloudWatch).
- A portfolio-grade repository and this handbook as supporting documentation for interviews and demos.

---

2. Architecture Overview

2.1 Component Inventory

Component	Role
Developer	Writes code, commits, opens pull requests
GitHub	Source of truth for application code, IaC, and manifests
GitHub Actions	Executes CI/CD pipeline (build, test, scan, push, deploy)
Terraform	Provisions and manages all AWS infrastructure
Amazon ECR	Private Docker registry storing versioned service images
Amazon EKS	Managed Kubernetes cluster running all microservices
Amazon RDS (PostgreSQL)	Managed relational database shared by all services
Amazon ElastiCache (Redis)	Managed in-memory cache for hot data and sessions
Route 53	DNS management for the platform's domain
CloudFront	CDN — caches static assets, terminates edge traffic, reduces latency
AWS WAF	Web Application Firewall — filters malicious traffic before it reaches the ALB
Application Load Balancer (ALB)	Layer 7 load balancer routing traffic into the EKS cluster
AWS Load Balancer Controller	Kubernetes controller that provisions/manages the ALB from Ingress resources
ACM	Issues and manages free TLS certificates for HTTPS
CloudWatch	AWS-native logs, metrics, and alarms
Prometheus	Pulls and stores time-series metrics from each microservice
Grafana	Visualizes Prometheus/CloudWatch metrics in dashboards
AWS Secrets Manager	Stores database credentials and API keys securely

2.2 Why Each Component Exists

- **GitHub Actions** is the automation backbone — it removes manual, error-prone deployment steps and enforces that every change passes quality and security gates before reaching production.
- **Terraform** guarantees that infrastructure is version-controlled, reviewable, and reproducible — critical when a team member leaves or infra needs to be rebuilt in a new AWS account.
- **ECR** exists so container images never have to be pulled from the public internet in production; it is private, IAM-controlled, and integrates natively with EKS.
- **EKS** provides self-healing, auto-scaling orchestration for the five microservices, replacing manual server management.
- **RDS PostgreSQL** offloads database operations (backups, patching, failover) to AWS instead of the team managing a self-hosted database.
- **ElastiCache Redis** reduces database load and latency for frequently accessed data (e.g., product catalog reads).
- **CloudFront + Route 53 + ACM** together deliver a fast, secure (HTTPS), custom-domain public entry point.
- **WAF** blocks common web exploits (SQL injection, XSS, bot traffic) before they reach the application layer.
- **Secrets Manager** ensures no credentials are hardcoded in code, images, or Kubernetes manifests in plaintext.
- **Prometheus/Grafana/CloudWatch** give the team both application-level and infrastructure-level visibility, enabling proactive incident response.

2.3 How Components Connect

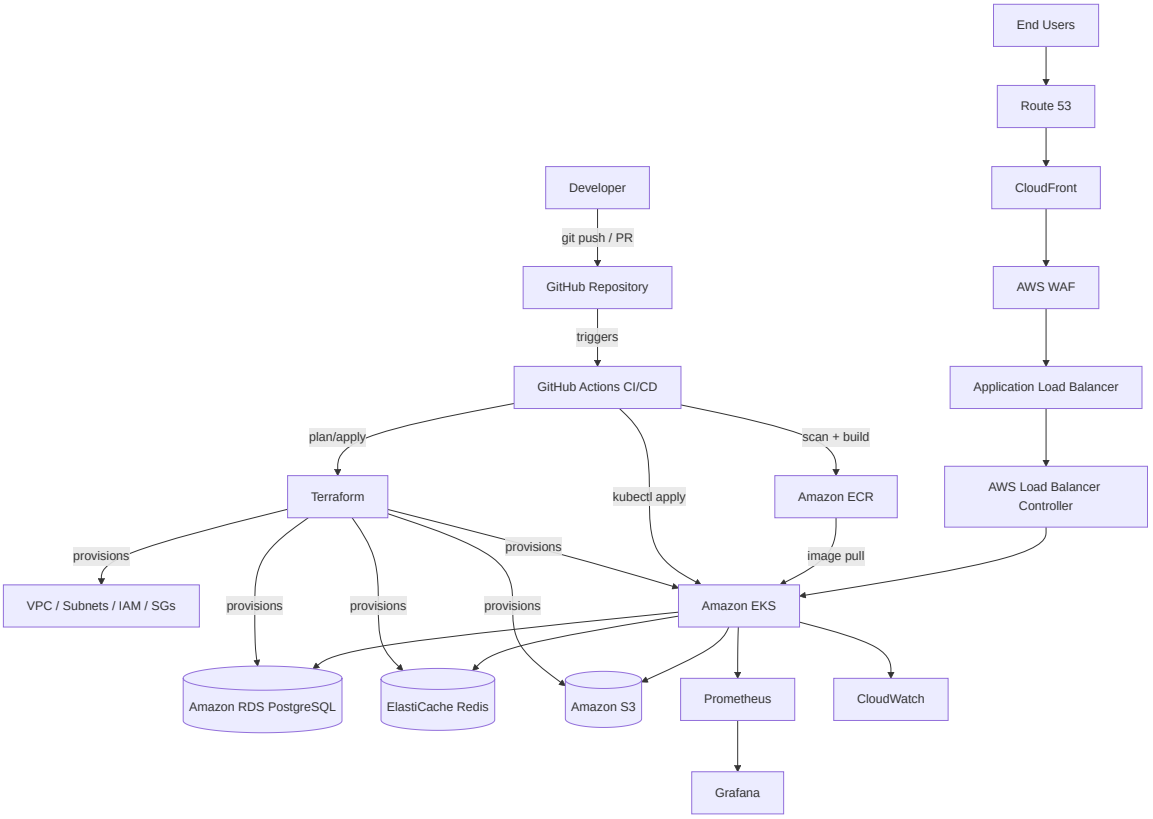


Diagram 1

2.4 Request Flow (End-to-End)

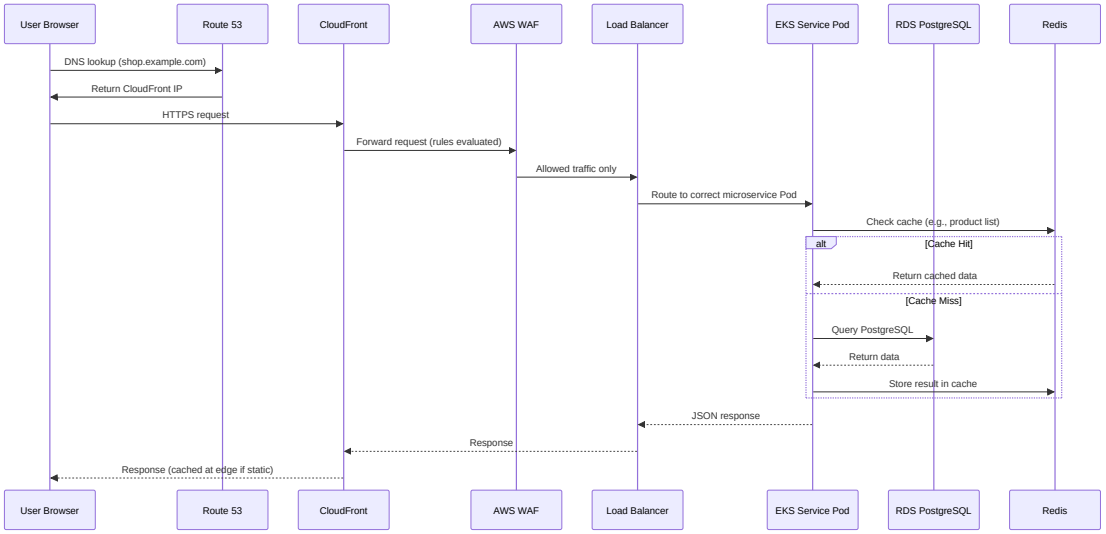


Diagram 2

2.5 High-Level Network Zones

Zone	Contents	Internet Access
Public Subnets	ALB, NAT Gateway	Yes (inbound via ALB, outbound via IGW)

Zone	Contents	Internet Access
Private Subnets	EKS worker nodes, application Pods	No direct inbound; outbound via NAT Gateway only
Data Subnets	RDS, ElastiCache	None — reachable only from private subnets via Security Groups

3. Microservices

All services follow the same conventions to keep the codebase predictable. Below is the full specification for **Product Service** as the reference implementation; the remaining four services (Cart, Inventory, Order, Notification) follow the identical pattern, with their differences called out afterward.

3.1 Product Service (Reference Implementation)

Purpose

Owns the product catalog: creation, retrieval, update, and deletion of product listings, categories, and pricing.

Responsibilities

- Serve product listing and detail data to the frontend.
- Provide product data to the Cart and Order services via internal REST calls.
- Cache frequently requested product data in Redis.

Folder Structure

```
product-service/
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── models.py
│   ├── routes/
│   │   └── product_routes.py
│   ├── services/
│   │   └── product_service.py
│   ├── schemas/
│   │   └── product_schema.py
│   └── utils/
│       ├── cache.py
│       └── logger.py
├── tests/
│   ├── unit/
│   └── integration/
├── migrations/
├── Dockerfile
├── requirements.txt
├── .env.example
└── wsgi.py
```

REST APIs

Method	Endpoint	Description	Auth
GET	`/api/v1/products`	List products (paginated, filterable)	Public
GET	`/api/v1/products/<id>`	Get single product	Public
POST	`/api/v1/products`	Create product	Admin
PUT	`/api/v1/products/<id>`	Update product	Admin
DELETE	`/api/v1/products/<id>`	Delete product	Admin
GET	`/health`	Liveness/readiness probe	Internal
GET	`/metrics`	Prometheus metrics	Internal

Dependencies

Flask, Flask-SQLAlchemy, Flask-Migrate, psycopg2-binary, redis-py, marshmallow, gunicorn, prometheus-flask-exporter.

Database Tables

Table	Key Columns
`products`	id, name, description, price, category_id, stock_ref, created_at
`categories`	id, name

Inter-Service Communication

- **Cart Service → Product Service**: `GET /api/v1/products/<id>` to validate product existence/price when adding to cart.
- **Order Service → Product Service**: reads product price at order-creation time.

Health Endpoint

`GET /health` returns `200 OK` with `{"status": "up", "db": "connected", "cache": "connected"}`. Used by Kubernetes liveness and readiness probes.

Dockerfile (reference)

```
FROM python:3.12-slim AS base
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd -m appuser && chown -R appuser /app
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=3s CMD curl -f http://localhost:5000/health || exit 1
CMD ["gunicorn", "-b", "0.0.0.0:5000", "-w", "4", "wsgi:app"]
```

Environment Variables

Variable	Description
`DATABASE_URL`	PostgreSQL connection string (from Secrets Manager)
`REDIS_URL`	Redis connection string
`FLASK_ENV`	`production` / `development`
`LOG_LEVEL`	`INFO` / `DEBUG`

Logging

Structured JSON logging (via `python-json-logger`) to stdout, collected by CloudWatch Container Insights / Fluent Bit.

Validation

Request bodies validated with Marshmallow schemas; invalid input returns `400` with a structured error payload.

Testing

- **Unit tests:** business logic in `services/`, mocked DB/cache.
- **Integration tests:** spin up a test PostgreSQL container, verify full request/response cycle.
- Minimum coverage target: 80%.

3.2 Cart Service

Purpose: Manages per-user shopping carts. **Key tables:** `carts`, `cart_items`. **Calls out to:** Product Service (validate product + price). **Notable endpoints:** `POST /api/v1/cart/items`, `DELETE /api/v1/cart/items/<id>`, `GET /api/v1/cart`.

3.3 Inventory Service

Purpose: Tracks stock levels per product/warehouse. **Key tables:** `inventory`, `stock_movements`. **Calls out to:** none (source of truth); is called by Order Service to reserve/decrement stock. **Notable endpoints:** `GET /api/v1/inventory/<product_id>`, `POST /api/v1/inventory/reserve`.

3.4 Order Service

Purpose: Creates and manages orders; orchestrates Product, Inventory, and Notification calls. **Key tables:** `orders`, `order_items`. **Calls out to:** Product Service (price), Inventory Service (reserve stock), Notification Service (order confirmation). **Notable endpoints:** `POST /api/v1/orders`, `GET /api/v1/orders/<id>`, `GET /api/v1/orders?user_id=`.

3.5 Notification Service (Mock)

Purpose: Simulates order/email notifications for demo purposes (logs to stdout/CloudWatch instead of sending real emails). **Key tables:** `notifications_log`. **Notable endpoints:** `POST /api/v1/notify`.

3.6 Cross-Service Communication Diagram

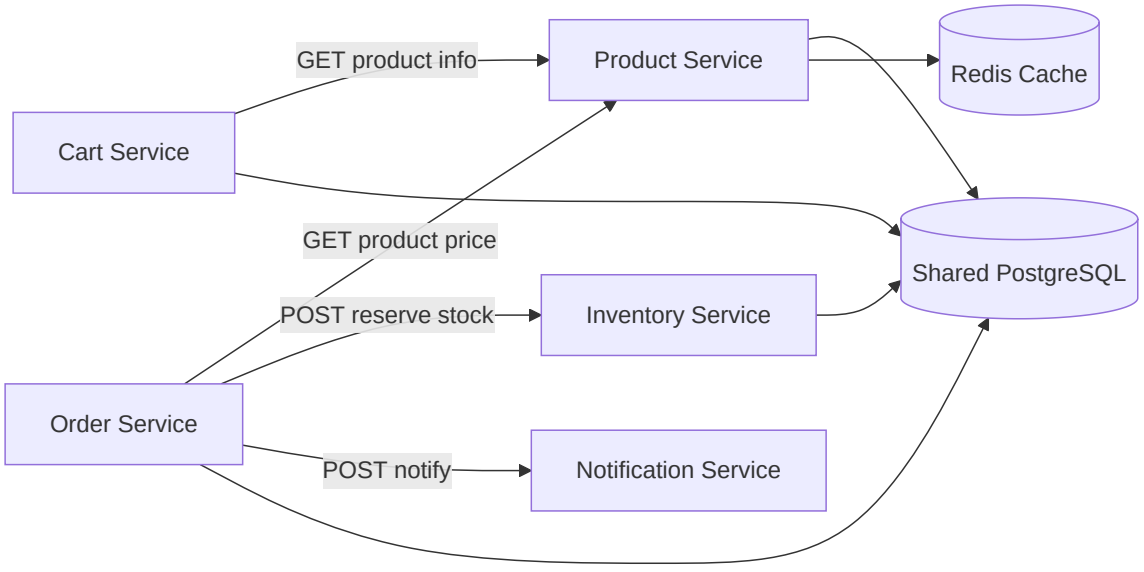


Diagram 3

4. Development Standards

4.1 Folder Structure (Monorepo Root)

```
ecommerce-platform/
├── services/
│   ├── product-service/
│   ├── cart-service/
│   ├── inventory-service/
│   ├── order-service/
│   └── notification-service/
├── infrastructure/
│   └── terraform/
├── k8s/
│   ├── base/
│   └── overlays/
├── .github/
│   └── workflows/
├── docs/
└── README.md
```

4.2 Naming Conventions

Item	Convention	Example
Python files/modules	snake_case	`product_routes.py`
Classes	PascalCase	`ProductService`
Functions/variables	snake_case	`get_product_by_id`
Environment variables	UPPER_SNAKE_CASE	`DATABASE_URL`
Docker images	`<service>:<git-sha>`	`product-service:a1b2c3d`
Kubernetes resources	`<service>-<resource>`	`product-service-deployment`
Terraform resources	`<project>-<env>-<resource>`	`ecom-prod-eks-cluster`

4.3 Git Convention & Branch Strategy

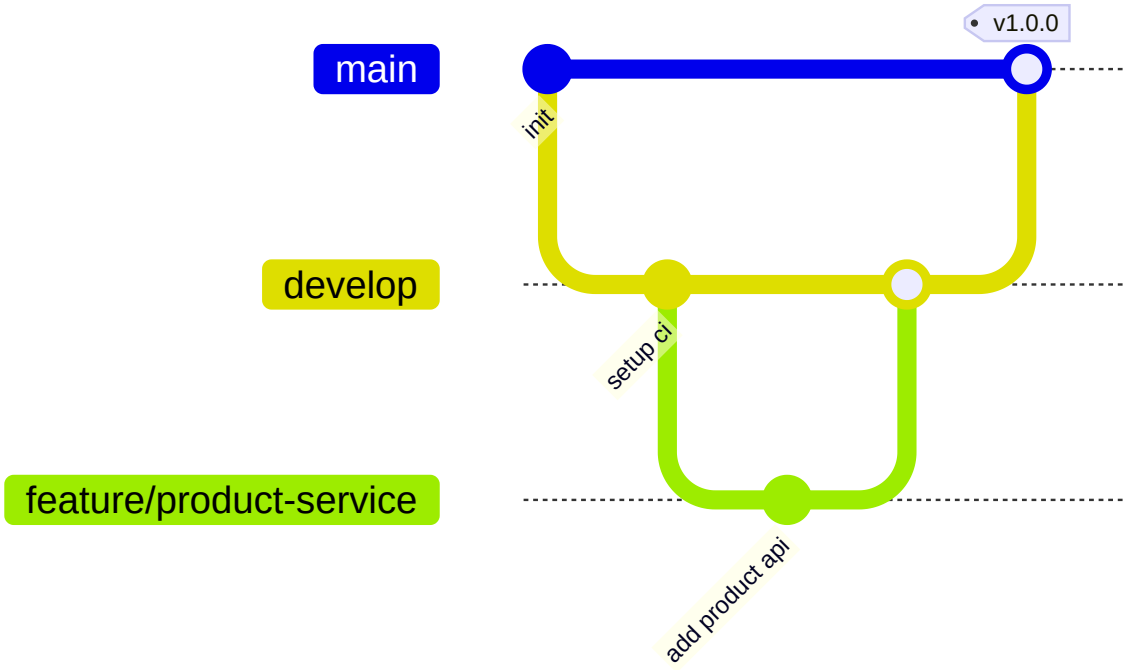


Diagram 4

Branch	Purpose
`main`	Production-ready; protected; deploys to prod on merge
`develop`	Integration branch; deploys to staging
`feature/*`	One branch per feature/service task
`hotfix/*`	Emergency production fixes

4.4 Commit Message Convention (Conventional Commits)

```
<type>(<scope>): <short summary>

feat(product-service): add category filter to product listing
fix(order-service): correct stock reservation race condition
chore(ci): bump trivy action to v0.20
docs(readme): update deployment instructions
```

Types: `feat`, `fix`, `chore`, `docs`, `test`, `refactor`, `ci`, `perf`.

4.5 Code Reviews

- Minimum **1 approving review** before merge to `develop`.
- Minimum **2 approving reviews** before merge to `main`.
- All CI checks (lint, tests, security scans) must pass — no exceptions.
- PR template requires: description, related issue, testing performed, screenshots (if UI-related).

4.6 Python Style Guide

- Formatter: **Black** (line length 88).

- Linter: **Flake8**.
- Import sorting: **isort**.
- Type hints required on all public function signatures.
- Docstrings: Google style.

4.7 API Standards

- All endpoints versioned under `/api/v1/`.
- JSON request/response bodies only.
- Standard error format:

```
{
  "error": "ValidationError",
  "message": "price must be a positive number",
  "status": 400
}
```

- Pagination via `?page=&per_page=` with `X-Total-Count` response header.

4.8 Swagger / OpenAPI

Each service exposes `GET /api/v1/docs` via `flasgger` or `apispec`, auto-generated from route docstrings/schemas. A merged, top-level API gateway doc is generated in Phase 13.

4.9 Exception Handling

Centralized Flask error handlers per service:

```
@app.errorhandler(ValidationError)
def handle_validation_error(e):
    return jsonify(error="ValidationError", message=str(e)), 400
```

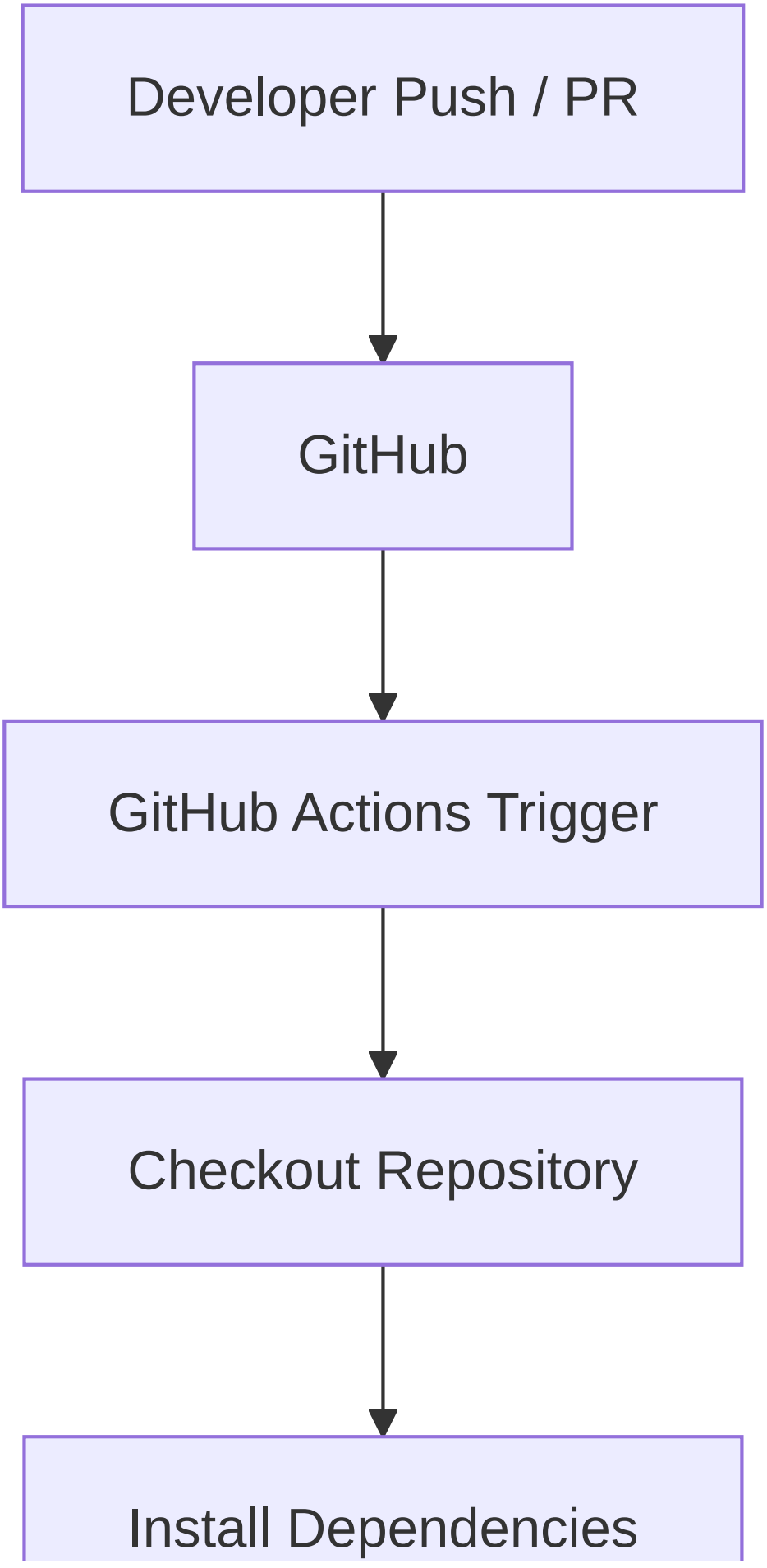
4.10 Configuration Management

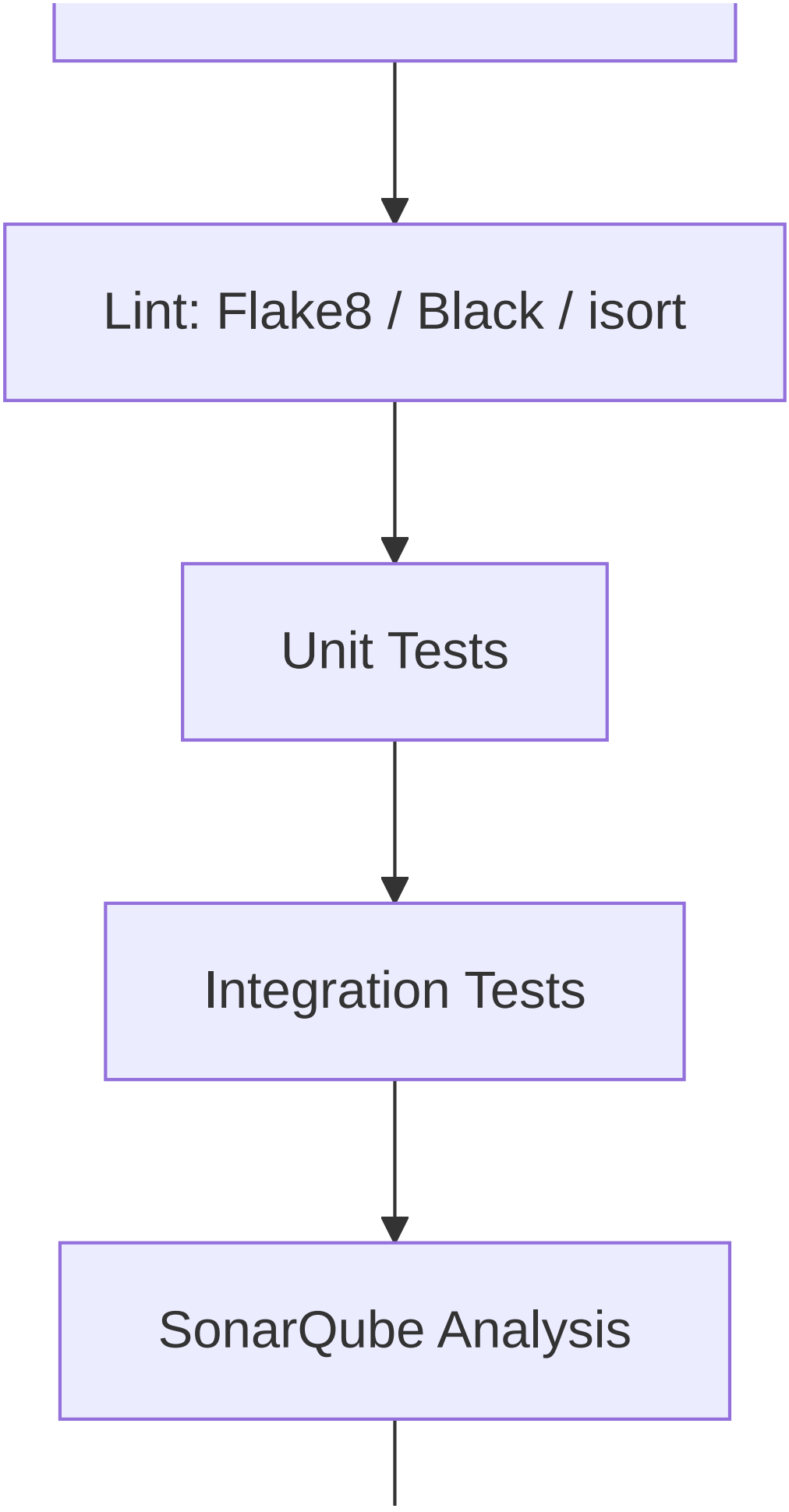
- All configuration via environment variables, never hardcoded.
- Local development uses `.env` files (git-ignored) based on `.env.example`.
- Production secrets are injected via Kubernetes Secrets sourced from AWS Secrets Manager (see Section 8).

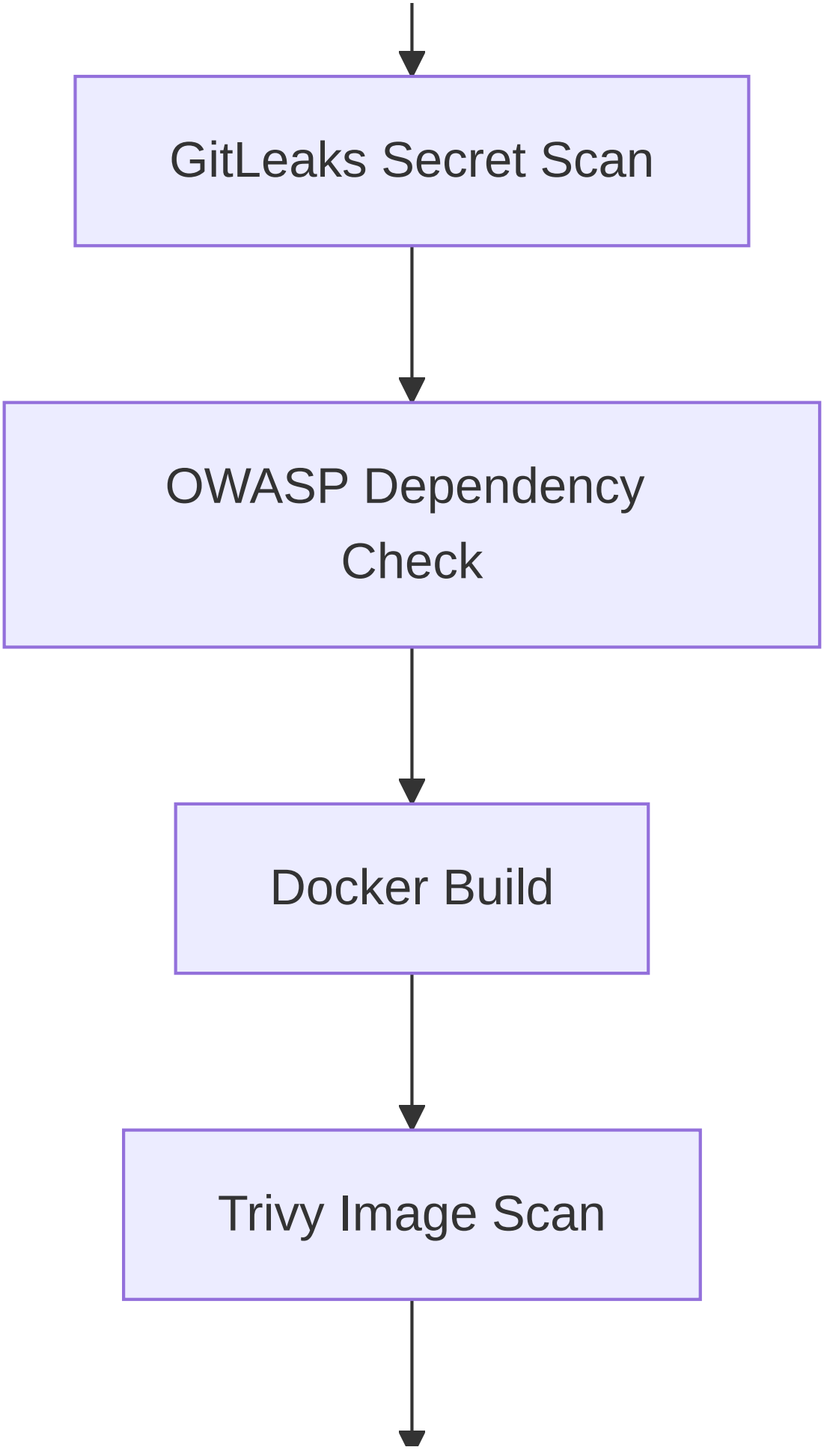
---

5. DevSecOps Pipeline

5.1 Pipeline Diagram


Developer Push / PR
GitHub
GitHub Actions Trigger
Checkout Repository
Install Dependencies


Lint: Flake8 / Black / isort
Unit Tests
Integration Tests
SonarQube Analysis


GitLeaks Secret Scan
OWASP Dependency Check
Docker Build
Trivy Image Scan

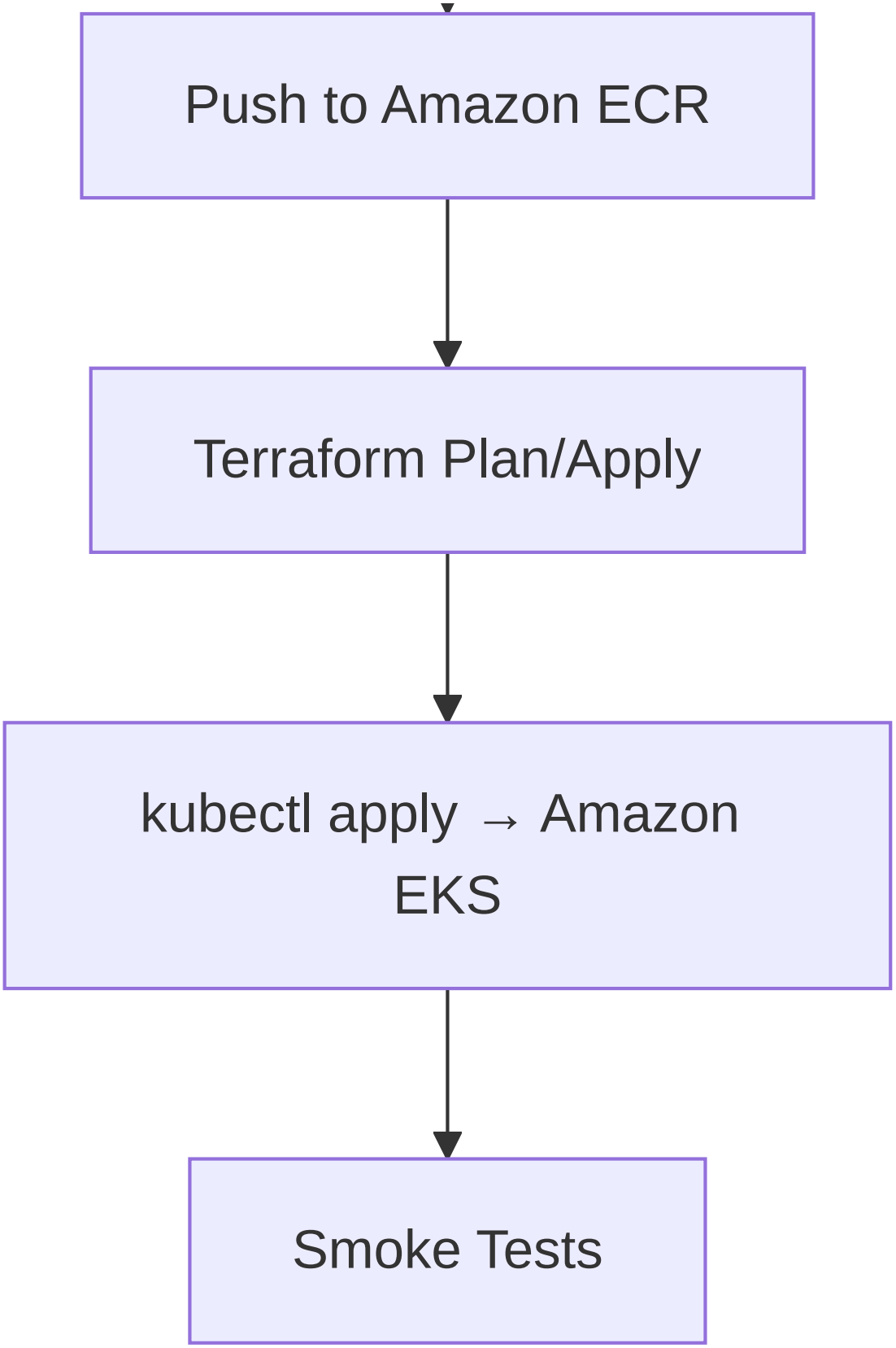


Diagram 5

5.2 Stage-by-Stage Reference

Checkout Repository

- **Purpose:** Pull the exact commit under test into the runner.

- **Input:** Git ref (branch/PR SHA).
- **Output:** Repository checked out to `$GITHUB_WORKSPACE`.
- **Tools:** `actions/checkout@v4`.
- **Failure scenarios:** Invalid token/permissions, submodule failures.
- **Best practice:** Pin action versions by SHA, not just tag.

Install Dependencies

- **Purpose:** Reproducible dependency install per service.
- **Input:** `requirements.txt` (hashed/locked).
- **Output:** Populated virtualenv or pip cache.
- **Tools:** `pip`, GitHub Actions cache.
- **Failure scenarios:** Version conflicts, yanked packages.
- **Best practice:** Pin exact versions; use `pip-audit` alongside OWASP DC.

Lint

- **Purpose:** Enforce consistent, readable, bug-resistant code style.
- **Tools:** Flake8, Black `--check`, isort `--check`.
- **Expected result:** Zero violations.
- **Failure scenario:** Style drift blocks merge — developer runs `black .` locally to fix.

Unit Tests

- **Purpose:** Validate isolated business logic.
- **Tools:** `pytest`, `pytest-cov`.
- **Expected result:** 100% pass, ≥80% coverage.
- **Failure scenario:** Broken logic — pipeline halts before any deployment risk.

Integration Tests

- **Purpose:** Validate service behavior against a real (ephemeral) PostgreSQL/Redis instance.
- **Tools:** `pytest`, Docker Compose service containers in the GitHub Actions runner.
- **Failure scenario:** Schema drift between models and migrations.

SonarQube Analysis

- **Purpose:** Static Application Security Testing (SAST) + code quality/technical debt scoring.
- **Input:** Source code + coverage report.
- **Output:** Quality Gate pass/fail, dashboard report.
- **Failure scenario:** New code introduces critical vulnerability or coverage drop → Quality Gate fails, PR blocked.

GitLeaks Scan

- **Purpose:** Detect committed secrets (API keys, passwords, tokens) in the diff and full history.
- **Failure scenario:** Any match fails the build immediately — secret must be revoked and purged from history.

OWASP Dependency Check

- **Purpose:** Software Composition Analysis (SCA) — flags known CVEs in third-party Python packages.
- **Output:** HTML/JSON report of vulnerable dependencies with severity.
- **Failure scenario:** Critical/High CVE with no available patch → team documents a risk acceptance or pins a patched version.

Docker Build

- **Purpose:** Build the immutable, versioned container image for the service.
- **Output:** Tagged local image (`service:$GIT_SHA`).

- **Best practice:** Multi-stage builds, non-root user, `.dockerignore`.

Trivy Image Scan

- **Purpose:** Scan the built image for OS and library-level vulnerabilities (image-layer SCA).
- **Failure scenario:** Critical vulnerability in base image → block push, switch base image or wait for patch.

Push to Amazon ECR

- **Purpose:** Publish the scanned, approved image to the private registry.
- **Tools:** `aws-actions/amazon-ecr-login`, `docker push`.
- **Best practice:** Tag with Git SHA **and** semantic version on release branches; enable ECR image scanning as a second layer.

Terraform Plan/Apply

- **Purpose:** Reconcile AWS infrastructure with the desired state defined in code.
- **Trigger:** Only on `main`, and only when files under `infrastructure/terraform/**` change.
- **Best practice:** `plan` runs on every PR and is posted as a comment; `apply` requires manual approval gate.

kubectl apply → EKS

- **Purpose:** Roll out the new image/manifests to the cluster using a rolling update strategy.
- **Best practice:** Use `kubectl rollout status` to verify success and auto-rollback on failure.

Smoke Tests

- **Purpose:** Verify the deployed service responds correctly in the real environment (`/health` and one core endpoint).
- **Failure scenario:** Triggers automatic rollback to the previous image tag.

5.3 Pipeline Failure Policy

Stage Fails	Pipeline Behavior
Lint / Unit / Integration Tests	Block merge, no deployment
SonarQube Quality Gate	Block merge
GitLeaks	Block merge, mandatory secret rotation
OWASP Dependency Check (Critical)	Block merge until resolved/waived
Trivy (Critical)	Block image push
Terraform Apply error	Halt, alert on-call, no partial rollout
Smoke Test failure	Auto-rollback to last known-good image

6. Infrastructure (Terraform)

Resource	Purpose	Why It's Required
VPC	Isolated network boundary for the whole platform	Prevents exposure of internal resources to the public internet by default
Public Subnets	Host internet-facing resources (ALB, NAT)	Required for ALB to receive public traffic
Private Subnets	Host EKS worker nodes and Pods	Keeps compute out of direct internet reach
Internet Gateway	Allows public subnet resources to reach/be reached from the internet	Needed for ALB and NAT outbound access
NAT Gateway	Lets private subnet resources reach the internet outbound (e.g., pull packages)	Required for nodes to pull container images, OS updates
Route Tables	Define traffic routing per subnet	Enforces the public/private network boundary
IAM Roles & Policies	Least-privilege access for EKS, GitHub Actions (OIDC), nodes	Prevents over-privileged credentials
Security Groups	Stateful firewalls per resource (EKS nodes, RDS, Redis)	Enforces that only expected ports/sources communicate
Amazon EKS	Managed Kubernetes control plane	Removes the operational burden of running Kubernetes masters
Amazon ECR	Private container registry	Secure, IAM-gated image storage integrated with EKS
Amazon RDS (PostgreSQL)	Managed relational database	Automated backups, patching, Multi-AZ failover
ElastiCache (Redis)	Managed in-memory cache	Reduces DB load, improves latency
S3	Object storage (static assets, Terraform state, ALB logs)	Durable, versioned storage
CloudFront	CDN	Global edge caching, reduced latency, HTTPS termination
Route 53	DNS	Maps custom domain to CloudFront/ALB
ACM	TLS certificate issuance	Free, auto-renewing HTTPS certificates
AWS WAF	Layer 7 firewall	Blocks OWASP Top 10 style attacks at the edge
CloudWatch	Logs, metrics, alarms	Native AWS observability and alerting

6.1 Terraform Module Layout

```
infrastructure/terraform/
├── modules/
│   ├── vpc/
│   ├── eks/
│   ├── ecr/
│   ├── rds/
│   ├── redis/
│   ├── s3/
│   ├── cloudfront/
│   ├── route53/
│   ├── waf/
│   └── iam/
├── environments/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── backend.tf
│   └── prod/
│       ├── main.tf
│       ├── variables.tf
│       └── backend.tf
└── README.md
```

6.2 Remote State

- Backend: **S3** (state file) + **DynamoDB** (state locking).
- One state file per environment (`dev` , `prod`) to prevent blast-radius overlap.

6.3 Example: EKS Module Call

```
module "eks" {
  source          = "./modules/eks"
  cluster_name    = "ecom-${var.environment}-eks"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnet_ids
  node_instance_type = "t3.medium"
  node_desired_size  = 2
  node_min_size      = 2
  node_max_size      = 5
}
```

6.4 Terraform Workflow in CI/CD

Diagram 6

7. Kubernetes

7.1 Core Concepts Used

Concept	Purpose
Namespace	Logical isolation per environment (`ecom-dev` , `ecom-prod`)
Deployment	Declarative rollout/rollback management for stateless Pods
Pod	Smallest deployable unit; runs one service container
ReplicaSet	Maintains the desired number of Pod replicas (managed by Deployment)
Service	Stable internal DNS name + load balancing across Pods
Ingress	HTTP(S) routing rules exposed externally via the ALB Controller
ConfigMap	Non-secret configuration (log level, feature flags)
Secret	Sensitive configuration (DB creds), sourced from Secrets Manager
AWS Load Balancer Controller	Watches Ingress resources and provisions/updates the real ALB

7.2 Namespace Layout

```
ecom-dev/
ecom-staging/
ecom-prod/
monitoring/   (Prometheus + Grafana)
```

7.3 Example Deployment Manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
  namespace: ecom-prod
spec:
  replicas: 3
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
    spec:
      containers:
        - name: product-service
          image: <account_id>.dkr.ecr.us-east-1.amazonaws.com/product-service:GIT_SHA
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: product-service-config
            - secretRef:
                name: product-service-secrets
          readinessProbe:
            httpGet: { path: /health, port: 5000 }
            initialDelaySeconds: 5
          livenessProbe:
            httpGet: { path: /health, port: 5000 }
            initialDelaySeconds: 15
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "500m", memory: "256Mi" }
```

7.4 Ingress (via AWS Load Balancer Controller)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecom-ingress
  namespace: ecom-prod
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
spec:
  rules:
    - host: api.shop.example.com
      http:
        paths:
          - path: /api/v1/products
            pathType: Prefix
            backend:
              service: { name: product-service, port: { number: 80 } }
          - path: /api/v1/cart
            pathType: Prefix
            backend:
              service: { name: cart-service, port: { number: 80 } }
```

7.5 How Deployment Works

1. CI pipeline pushes a new image to ECR.
2. CI runs `kubectl set image deployment/product-service product-service=<new-image>`.
3. Kubernetes performs a **rolling update**: spins up new Pods, waits for readiness probes to pass, then terminates old Pods gradually.

4. If new Pods fail readiness, the rollout halts and can be rolled back with `kubectl rollout undo`.

7.6 How Scaling Works

- **Horizontal Pod Autoscaler (HPA)** scales replicas based on CPU/memory utilization.
- **Cluster Autoscaler** adds/removes EKS worker nodes based on unschedulable Pods.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: product-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: product-service
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
```

7.7 Networking Model

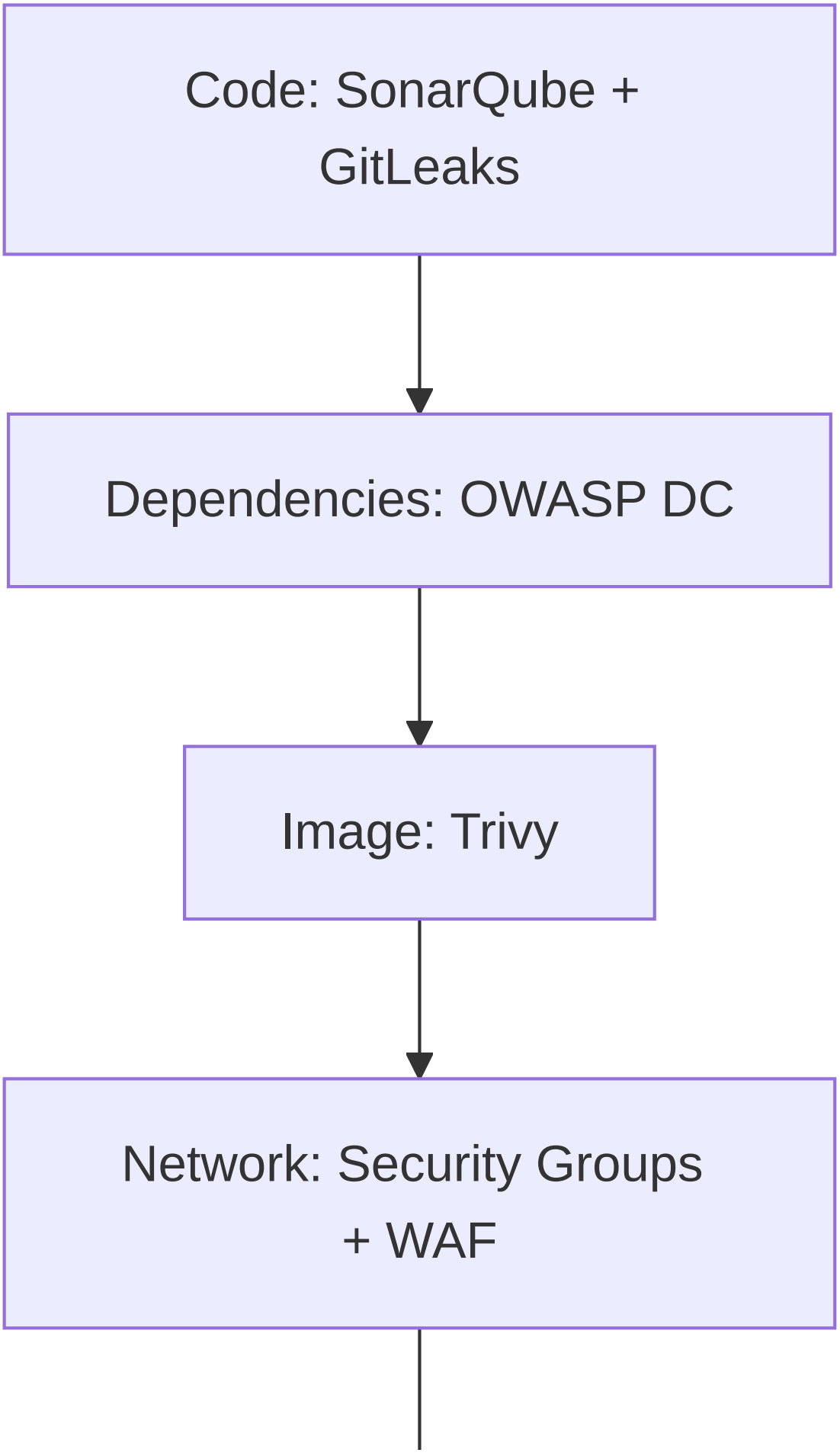
- Each Pod gets its own IP (via the AWS VPC CNI plugin).
- Services provide stable ClusterIP DNS names (`product-service.ecom-prod.svc.cluster.local`).
- Network Policies (optional hardening) restrict which services can talk to which.

---

8. Security

Tool/Control	Category	What It Does
SonarQube	SAST	Scans source code for bugs, code smells, and known vulnerability patterns
GitLeaks	Secret Scanning	Scans commits/history for hardcoded credentials
OWASP Dependency Check	SCA	Flags known CVEs in third-party dependencies
Trivy	Container Scanning	Scans Docker images for OS/library vulnerabilities and misconfigurations
AWS Secrets Manager	Secrets Storage	Centrally stores and rotates DB credentials, API keys
AWS KMS	Encryption	Manages encryption keys for RDS, S3, Secrets Manager
IAM	Access Control	Enforces least-privilege permissions for humans and services
Security Groups	Network Firewalling	Restricts traffic between components at the network layer
AWS WAF	Edge Protection	Blocks common web attacks (SQLi, XSS, bad bots) before they reach the app

8.1 Defense-in-Depth Layering


Code: SonarQube + GitLeaks
Dependencies: OWASP DC
Image: Trivy
Network: Security Groups + WAF

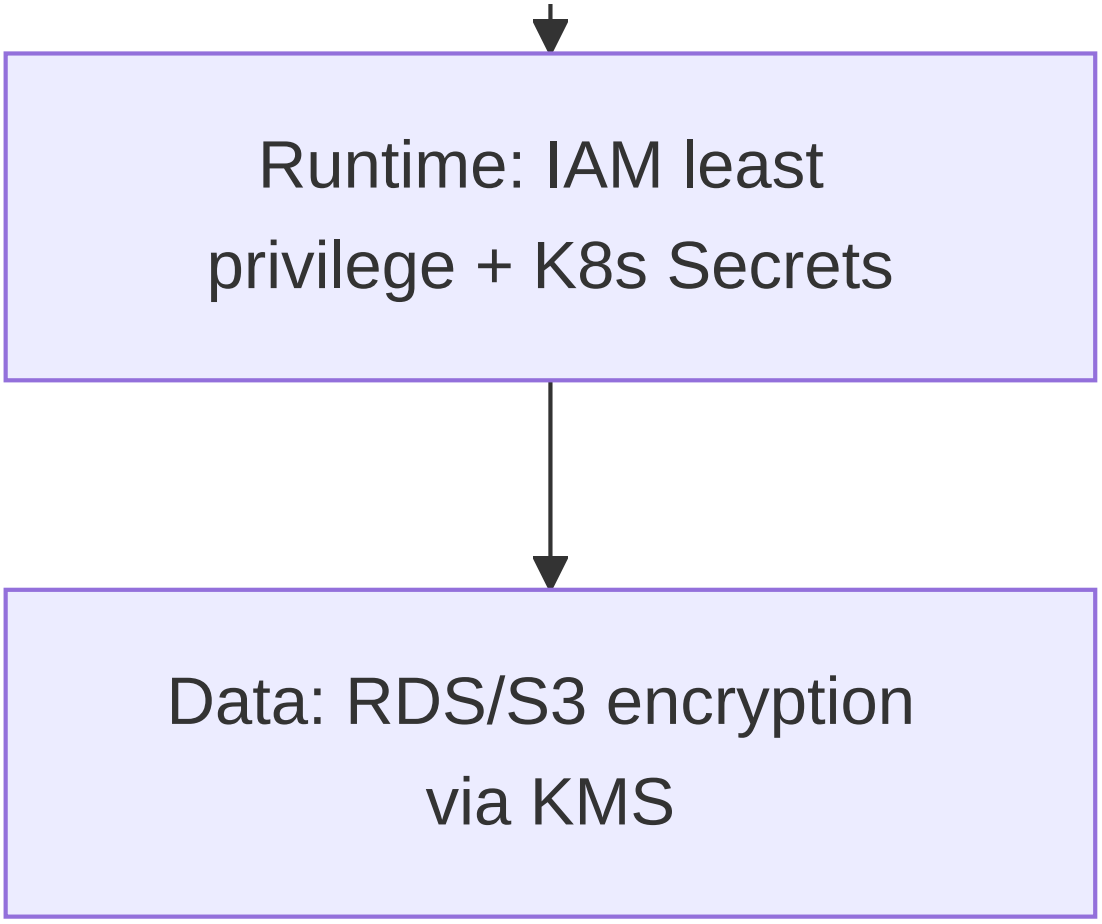


Diagram 7

8.2 Secrets Flow

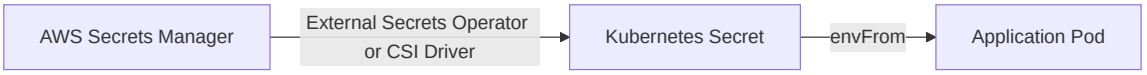


Diagram 8

- Secrets are **never** committed to Git, baked into Docker images, or stored as plain ConfigMaps.
- Rotation of DB credentials in Secrets Manager is automatically picked up by the External Secrets Operator on its sync interval.

8.3 Security Best Practices Checklist

- ☐ All containers run as non-root users.
- ☐ Base images pinned to specific digests, not `latest`.
- ☐ Security Groups follow least-privilege (RDS only accepts traffic from EKS node SG, not `0.0.0.0/0`).
- ☐ IAM roles scoped per service (IRSA — IAM Roles for Service Accounts).
- ☐ TLS enforced end-to-end (ACM cert on ALB; HTTPS-only listener).
- ☐ WAF rules include AWS Managed Rule Groups (Core Rule Set, Known Bad Inputs).
- ☐ All S3 buckets block public access by default.
- ☐ RDS encryption at rest enabled via KMS.
- ☐ Audit logging enabled (CloudTrail) for all infrastructure changes.

9. Monitoring

9.1 Tool Responsibilities

Tool	Scope
Prometheus	Application-level metrics scraped from each microservice's `/metrics` endpoint
Grafana	Dashboards/visualization for Prometheus and CloudWatch data sources
CloudWatch	Infrastructure-level metrics/logs (EKS nodes, RDS, ALB, Redis) and alarms

9.2 Metrics Collected

Category	Examples
Application	Request count, request duration, error rate (4xx/5xx), active DB connections
Kubernetes	Pod CPU/memory usage, Pod restarts, replica count vs desired
Infrastructure	RDS CPU/connections, Redis memory usage/evictions, ALB target health

9.3 Monitoring Architecture

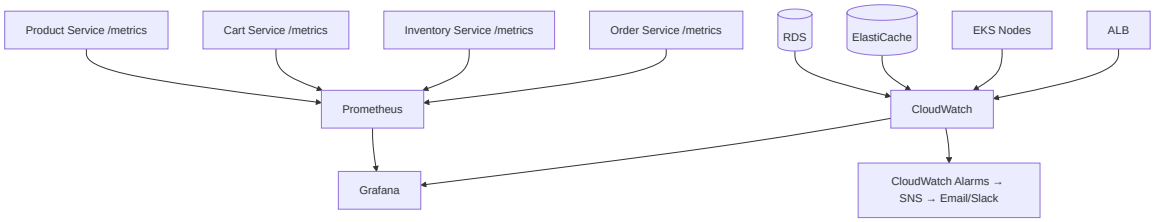


Diagram 9

9.4 Sample Dashboards

- **Service Overview:** request rate, p95 latency, error rate per service.
- **Infrastructure Health:** node CPU/memory, RDS connections, Redis hit ratio.
- **Business Metrics:** orders per hour, cart abandonment proxy (carts created vs orders created).

9.5 Alerting Rules (examples)

Alert	Condition	Action
High Error Rate	5xx rate > 5% for 5 minutes	Notify on-call via Slack
Pod CrashLoopBackOff	Restart count > 3 in 10 min	Notify on-call
RDS CPU High	CPU > 80% for 10 minutes	Notify + investigate scaling
Redis Memory Pressure	Evictions > 0 sustained	Notify + review cache TTLs

9.6 Health Checks

Every service exposes `GET /health`, consumed by: - Kubernetes liveness/readiness probes. - ALB target group health checks. - Smoke test stage in the CI/CD pipeline.

10. Team Organization

The project is organized into 13 execution phases (Section 11). This section defines how a 6–8 person team maps onto those phases.

Phase Group	Primary Owner(s)	Supporting Roles
Development (1)	Backend Engineers (x2-3)	QA Engineer
Containerization (2)	Backend Engineers	DevOps Engineer
Source Control (3)	All	Project Manager
CI Pipeline (4)	DevSecOps Engineer	Backend Engineers
Infrastructure as Code (5)	Terraform/Cloud Engineer	DevOps Engineer
Kubernetes Cluster (6)	Cloud/DevOps Engineer	DevSecOps Engineer
App Deployment (7)	DevOps Engineer	Backend Engineers
Networking (8)	Cloud Engineer	DevOps Engineer
Database Layer (9)	Cloud Engineer	Backend Engineers
Cache Layer (10)	Cloud Engineer	Backend Engineers
Monitoring (11)	DevOps Engineer	DevSecOps Engineer
Security (12)	DevSecOps Engineer	Cloud Engineer
Final Testing & Docs (13)	QA Engineer	All

Each phase entry below (Section 11) includes Objectives, Tasks, Deliverables, Dependencies, Required Knowledge, Estimated Duration, Team Members, Expected Output, Risks, and Definition of Done.

11. Phase-by-Phase Roadmap

Phase 1 — Development

- **Purpose:** Build the five Flask microservices with clean, testable, documented code.
- **Objectives:** Working REST APIs for all five services against a local/dev database.
- **Tasks:** Scaffold each service; implement models, routes, schemas; write unit + integration tests; write Dockerfiles (draft).
- **Inputs:** Requirements from Section 3.
- **Outputs:** Five runnable Flask services, passing test suites.
- **Dependencies:** None (starting phase).
- **Files Created:** `services/*/app/**`, `services/*/tests/**`, `requirements.txt` per service.
- **AWS Resources:** None yet.
- **GitHub Actions Changes:** None yet (pipeline built in Phase 4).
- **Terraform Changes:** None yet.
- **Testing Required:** Unit + integration tests, ≥80% coverage.
- **Documentation Required:** Per-service README with local run instructions.
- **Required Knowledge:** Python, Flask, SQLAlchemy, REST API design, Pytest.
- **Estimated Duration:** 2–3 weeks.
- **Team Members:** 2–3 Backend Engineers, 1 QA Engineer (test review).
- **Risks:** Scope creep on business logic; inconsistent conventions across services if not reviewed early.
- **Definition of Done:** All five services run locally, all tests pass, PR reviewed and merged to `develop`.

Phase 2 — Containerization

- **Purpose:** Package each service as a portable, reproducible Docker image.
- **Tasks:** Write production Dockerfiles (multi-stage, non-root); add `.dockerignore`; validate `docker-compose.yml` for local full-stack testing.
- **Outputs:** Five Docker images, buildable locally.
- **Dependencies:** Phase 1 complete.
- **Files Created:** `Dockerfile` per service, `docker-compose.yml`.
- **Testing Required:** `docker build` succeeds; container passes `/health` check.
- **Documentation Required:** “How to build and run locally with Docker” section in README.
- **Required Knowledge:** Docker, multi-stage builds, container security basics.
- **Estimated Duration:** 3–5 days.
- **Team Members:** Backend Engineers + DevOps Engineer.
- **Risks:** Image bloat from unnecessary layers; missing non-root user causing security scan failures later.
- **Definition of Done:** All five images build and run correctly via `docker-compose up`.

Phase 3 — Source Control

- **Purpose:** Establish the repository structure and collaboration rules.
- **Tasks:** Create GitHub repo (monorepo); set up branch protection on `main` / `develop`; add PR templates, CODEOWNERS.
- **Outputs:** Configured GitHub repository.
- **Dependencies:** None (can run in parallel with Phase 1).
- **Files Created:** `.github/PULL_REQUEST_TEMPLATE.md`, `CODEOWNERS`, `.gitignore`.
- **Documentation Required:** `CONTRIBUTING.md`.
- **Required Knowledge:** Git, GitHub branch protection, GitFlow-style branching.
- **Estimated Duration:** 1–2 days.

- **Team Members:** Project Manager + one senior engineer.
- **Risks:** Team not following branch convention without early enforcement.
- **Definition of Done:** Branch protection active; at least one team member has tested the PR flow end-to-end.

Phase 4 — CI Pipeline (GitHub Actions)

- **Purpose:** Automate lint, test, and security scanning on every push/PR.
- **Tasks:** Write reusable GitHub Actions workflow(s); configure SonarQube project; configure GitLeaks; configure OWASP Dependency Check; wire up Docker build + Trivy scan; push to ECR (after Phase 5 IAM/ECR exist).
- **Outputs:** `.github/workflows/ci.yml` running on every PR.
- **Dependencies:** Phase 1–3 complete; Phase 5 for ECR push step.
- **Files Created:** `.github/workflows/ci.yml`, `sonar-project.properties`, `.gitleaks.toml`.
- **AWS Resources:** OIDC IAM role for GitHub Actions (created in Phase 5).
- **Testing Required:** Deliberately break lint/tests/secret once to confirm pipeline correctly fails.
- **Documentation Required:** Pipeline stage reference (Section 5 of this handbook).
- **Required Knowledge:** GitHub Actions YAML, SAST/SCA tooling, Docker.
- **Estimated Duration:** 1–2 weeks.
- **Team Members:** DevSecOps Engineer (lead) + 1 Backend Engineer.
- **Risks:** False positives from security tools slowing the team; misconfigured quality gates blocking all merges.
- **Definition of Done:** A PR with intentionally bad code/secret is correctly blocked; a clean PR passes end-to-end.

Phase 5 — Infrastructure as Code

- **Purpose:** Provision all AWS infrastructure via Terraform.
- **Tasks:** Write VPC, IAM, EKS, ECR, RDS, Redis, S3, CloudFront, Route 53, ACM, WAF, CloudWatch modules; set up remote state (S3 + DynamoDB lock).
- **Outputs:** `terraform apply` successfully provisions the full environment.
- **Dependencies:** None technically, but should follow Phase 3 (repo exists).
- **Files Created:** `infrastructure/terraform/**` (see Section 6.1).
- **AWS Resources:** VPC, subnets, IGW, NAT, EKS, ECR, RDS, ElastiCache, S3, CloudFront, Route 53, ACM, WAF, CloudWatch.
- **GitHub Actions Changes:** Add `terraform-plan.yml` / `terraform-apply.yml` workflows.
- **Testing Required:** `terraform validate`, `terraform plan` reviewed, `terraform apply` in a `dev` environment first.
- **Documentation Required:** Section 6 of this handbook + module-level READMEs.
- **Required Knowledge:** Terraform, AWS networking, IAM.
- **Estimated Duration:** 2–3 weeks.
- **Team Members:** Terraform/Cloud Engineer (lead) + DevOps Engineer.
- **Risks:** Cost overruns if resources aren't destroyed after testing; IAM misconfiguration blocking EKS access.
- **Definition of Done:** `dev` environment fully provisioned and reachable; `terraform destroy` cleanly tears it down with no orphaned resources.

Phase 6 — Kubernetes Cluster

- **Purpose:** Prepare the EKS cluster for application workloads.
- **Tasks:** Configure `kubectl` / `aws eks update-kubeconfig`; install AWS Load Balancer Controller via Helm; create namespaces; set up ConfigMap/Secret patterns; install External Secrets Operator (Secrets Manager integration).
- **Outputs:** Cluster ready to receive Deployments.
- **Dependencies:** Phase 5 (EKS must exist).
- **Files Created:** `k8s/base/namespaces.yaml`, Helm values files.
- **AWS Resources:** IAM role for Load Balancer Controller (IRSA).
- **Testing Required:** Deploy a "hello world" Pod and confirm it's reachable via a test Ingress.
- **Documentation Required:** Section 7 of this handbook.

- **Required Knowledge:** Kubernetes, Helm, IRSA.
- **Estimated Duration:** 1 week.
- **Team Members:** Cloud/DevOps Engineer.
- **Risks:** Load Balancer Controller IAM permission issues (a very common EKS pitfall).
- **Definition of Done:** Test Ingress resolves to a working ALB URL.

Phase 7 — Application Deployment

- **Purpose:** Deploy all five microservices to EKS.
- **Tasks:** Write Deployment/Service/Ingress manifests per service; wire CI/CD `kubectl apply` step; verify rolling updates work.
- **Outputs:** Five services running in `ecom-dev` namespace, reachable via Ingress.
- **Dependencies:** Phases 4, 5, 6 complete.
- **Files Created:** `k8s/base/<service>/deployment.yaml`, `service.yaml`, `ingress.yaml`.
- **GitHub Actions Changes:** Add `cd-deploy.yml` invoking `kubectl apply` + `kubectl rollout status`.
- **Testing Required:** End-to-end smoke test hitting each service's `/health`.
- **Documentation Required:** Section 14 (Deployment Guide).
- **Required Knowledge:** Kubernetes manifests, kubectl, rolling updates.
- **Estimated Duration:** 1–2 weeks.
- **Team Members:** DevOps Engineer + Backend Engineers.
- **Risks:** CrashLoopBackOff due to missing env vars/secrets; readiness probe misconfiguration.
- **Definition of Done:** All five services `Running` with correct replica counts; Ingress routes correctly to each.

Phase 8 — Networking

- **Purpose:** Expose the platform securely to the public internet.
- **Tasks:** Configure Route 53 hosted zone; request ACM certificate; configure CloudFront distribution; attach WAF Web ACL; validate ALB listener rules (HTTP→HTTPS redirect).
- **Outputs:** `https://shop.example.com` resolves end-to-end.
- **Dependencies:** Phases 5, 7.
- **AWS Resources:** Route 53 records, ACM certificate, CloudFront distribution, WAF Web ACL.
- **Testing Required:** DNS resolution test, TLS certificate validity check, WAF rule test (e.g., simulated SQLi blocked).
- **Documentation Required:** Section 2.4 (Request Flow) validated against real traffic.
- **Required Knowledge:** DNS, TLS/ACM, CDN concepts, WAF rules.
- **Estimated Duration:** 3–5 days.
- **Team Members:** Cloud Engineer.
- **Risks:** Certificate validation delays; CloudFront cache serving stale API responses if misconfigured for dynamic content.
- **Definition of Done:** Public HTTPS URL works; HTTP requests are redirected to HTTPS; WAF actively blocks a test malicious payload.

Phase 9 — Database Layer

- **Purpose:** Stand up the shared production database and migrate schemas.
- **Tasks:** Provision RDS (done in Phase 5); run Alembic/Flask-Migrate migrations against RDS; configure automated backups and Multi-AZ (if budget allows).
- **Outputs:** Production-ready PostgreSQL instance with all service schemas applied.
- **Dependencies:** Phase 5.
- **Testing Required:** Connectivity test from within the cluster; migration rollback test.
- **Documentation Required:** Migration runbook.
- **Required Knowledge:** PostgreSQL, SQLAlchemy migrations.
- **Estimated Duration:** 3–5 days.
- **Team Members:** Cloud Engineer + Backend Engineers.
- **Risks:** Migration conflicts between services sharing one database — enforce a single migration owner/sequence.

- **Definition of Done:** All tables exist in RDS; each service connects successfully.

Phase 10 — Cache Layer

- **Purpose:** Reduce database load and latency for hot-path reads.
- **Tasks:** Provision ElastiCache Redis (done in Phase 5); implement caching in Product Service (and optionally session cache); define TTL/invalidation strategy.
- **Outputs:** Product listing reads served from cache on repeat requests.
- **Dependencies:** Phase 5, 9.
- **Testing Required:** Cache hit/miss verification; invalidation-on-update test.
- **Documentation Required:** Caching strategy notes in Product Service README.
- **Required Knowledge:** Redis, caching patterns (cache-aside).
- **Estimated Duration:** 3–5 days.
- **Team Members:** Backend Engineer + Cloud Engineer.
- **Risks:** Stale cache data after product updates if invalidation isn't wired correctly.
- **Definition of Done:** Measurable latency improvement on cached endpoints; invalidation verified.

Phase 11 — Monitoring

- **Purpose:** Establish full-stack observability.
- **Tasks:** Deploy Prometheus + Grafana via Helm into `monitoring` namespace; expose `/metrics` on all services; build Grafana dashboards; configure CloudWatch alarms + SNS notifications.
- **Outputs:** Live dashboards and alerting.
- **Dependencies:** Phase 7 (services deployed).
- **Files Created:** `k8s/monitoring/**`, Grafana dashboard JSON exports.
- **Testing Required:** Trigger a deliberate error spike and confirm alert fires.
- **Documentation Required:** Section 9 of this handbook + runbook links from alerts.
- **Required Knowledge:** Prometheus, Grafana, CloudWatch, PromQL basics.
- **Estimated Duration:** 1 week.
- **Team Members:** DevOps Engineer + DevSecOps Engineer.
- **Risks:** Alert fatigue from poorly tuned thresholds.
- **Definition of Done:** Dashboards show live data for all services; at least one working alert route to Slack/email.

Phase 12 — Security

- **Purpose:** Harden the platform beyond the CI/CD pipeline gates.
- **Tasks:** Finalize IAM least-privilege review; lock down Security Groups; enable KMS encryption everywhere applicable; integrate Secrets Manager via External Secrets Operator; review WAF rule effectiveness; run a full OWASP ZAP or similar DAST pass (stretch goal).
- **Outputs:** Documented security posture; closed findings from Phases 4/5/6 scans.
- **Dependencies:** Phases 5–11.
- **Testing Required:** Manual security review checklist (Section 8.3) fully checked off.
- **Documentation Required:** Section 8 of this handbook.
- **Required Knowledge:** AWS IAM, KMS, WAF, secure coding practices.
- **Estimated Duration:** 1 week.
- **Team Members:** DevSecOps Engineer (lead) + Cloud Engineer.
- **Risks:** Security hardening breaking existing connectivity if applied too aggressively without testing.
- **Definition of Done:** All items in the Security Best Practices Checklist (8.3) are checked.

Phase 13 — Final Testing & Documentation

- **Purpose:** Validate the full system and produce final deliverables.

- **Tasks:** Run smoke, API, end-to-end, and performance tests (k6/JMeter); finalize Swagger docs per service; finalize README, architecture diagrams, deployment guide, and this handbook.
- **Outputs:** Fully tested, fully documented, demo-ready platform.
- **Dependencies:** All previous phases.
- **Testing Required:** Full regression pass; load test at a defined target (e.g., 100 concurrent users).
- **Documentation Required:** All sections of this handbook finalized; final architecture diagram exported.
- **Required Knowledge:** Test automation (k6/JMeter), technical writing.
- **Estimated Duration:** 1–2 weeks.
- **Team Members:** QA Engineer (lead) + entire team.
- **Risks:** Performance issues discovered too late — mitigate by starting light load tests earlier, in parallel with Phase 11.
- **Definition of Done:** All tests pass; handbook complete; team can perform a live demo from a clean environment.

---

12. Team Responsibilities Matrix

Role	Responsibilities	Deliverables	Required Skills
Backend Engineer	Implement microservice business logic, APIs, tests	Working, tested Flask services	Python, Flask, SQLAlchemy, REST, Pytest
Cloud Engineer	Design and provision AWS infrastructure	Terraform modules, provisioned AWS resources	AWS (VPC, EKS, RDS, IAM), Terraform
DevOps Engineer	Build CI/CD, manage Kubernetes deployments	Working pipelines, Kubernetes manifests	GitHub Actions, Docker, Kubernetes, Helm
DevSecOps Engineer	Integrate and tune security tooling in the pipeline; own security posture	Passing security gates, security checklist	SAST/SCA/container scanning tools, IAM, secure SDLC
Terraform Engineer (can overlap with Cloud Engineer)	Own IaC modules, state management, environment parity	Reusable Terraform modules, environment configs	Terraform, AWS, module design
QA Engineer	Write/execute test plans, coordinate final validation	Test reports, coverage reports, load test results	Pytest, API testing tools, k6/JMeter
Project Manager	Coordinate phases, track dependencies/timeline, manage risk	Timeline, status reports, risk log	Project coordination, Git workflow literacy

13. Project Timeline

Assumes a team of 6–8 members working across a ~14-week semester, with several phases run in parallel.

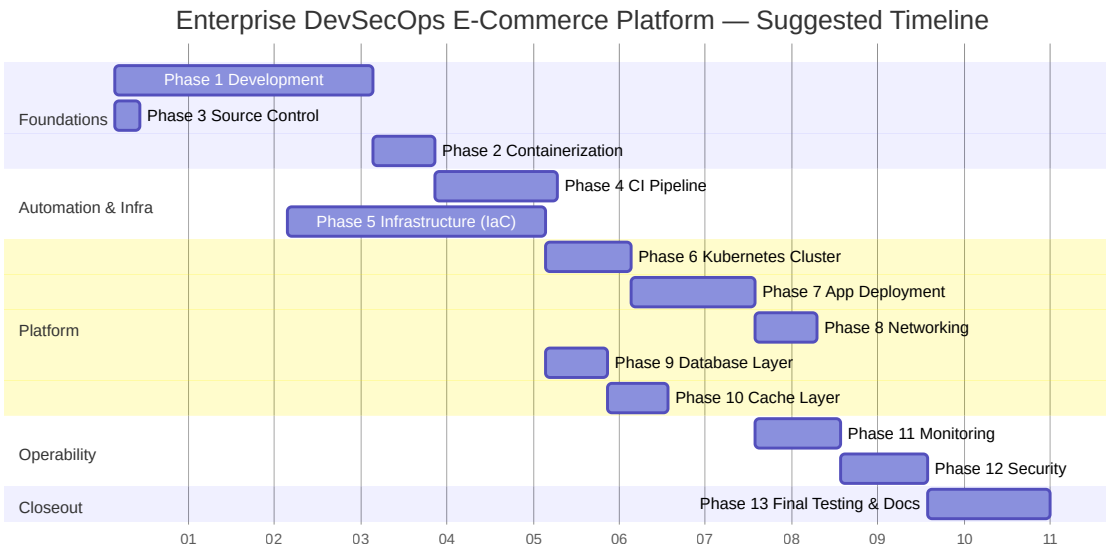


Diagram 10

13.1 Critical Path

`Phase 1 (Development) → Phase 2 (Containerization) → Phase 4 (CI Pipeline) → Phase 5 (Infrastructure) → Phase 6 (K8s Cluster) → Phase 7 (App Deployment) → Phase 11 (Monitoring) → Phase 12 (Security) → Phase 13 (Final Testing & Docs)`

Phases 3, 8, 9, and 10 can run in parallel with the critical path and do not extend the overall timeline if properly staffed.

13.2 Milestones

Milestone	Target Week	Success Criteria
M1: All services runnable locally	Week 3	`docker-compose up` brings up full stack
M2: CI pipeline green	Week 5	PR triggers full lint/test/scan pipeline successfully
M3: Dev infrastructure live	Week 7	`terraform apply` succeeds; EKS reachable
M4: First production-like deployment	Week 9	All 5 services running on EKS, reachable via Ingress
M5: Public HTTPS endpoint	Week 10	`https://shop.example.com` resolves and works
M6: Monitoring & Security signed off	Week 12	Dashboards live; security checklist 100% complete
M7: Final demo ready	Week 14	Full regression + load test pass; handbook finalized

14. Deployment Guide

14.1 Deployment Flow

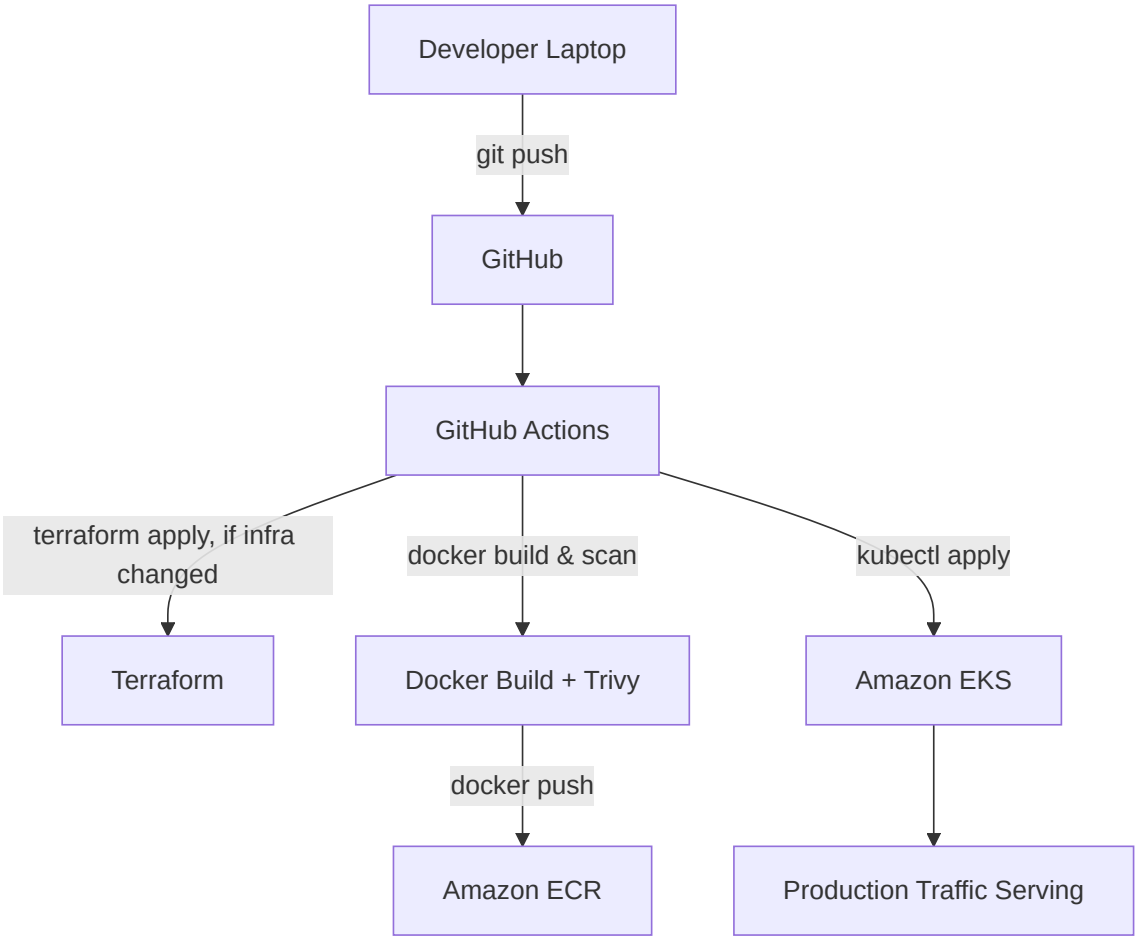


Diagram 11

14.2 Step-by-Step: From Zero to Production

1. **Local setup**

```
git clone https://github.com/<org>/ecommerce-platform.git
cd ecommerce-platform
docker-compose up --build
```

2. **Provision infrastructure (one-time per environment)**

```
cd infrastructure/terraform/environments/dev
terraform init
terraform plan -out=tfplan
terraform apply tfplan
```

3. **Configure kubectl**

```
aws eks update-kubeconfig --name ecom-dev-eks --region us-east-1
kubectl get nodes
```

4. **Install cluster add-ons**

```
helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system --set clusterName=ecom-dev-eks
```

5. **Push code → CI/CD takes over automatically** on merge to `develop` / `main`:
   - Lint → Test → SonarQube → GitLeaks → OWASP DC → Docker Build → Trivy → ECR push → `kubectl apply`.

6. **Verify deployment**

```
kubectl get pods -n ecom-prod
kubectl get ingress -n ecom-prod
curl https://api.shop.example.com/health
```

7. **Roll back if needed**

```
kubectl rollout undo deployment/product-service -n ecom-prod
```

14.3 Environment Promotion

Environment	Trigger	Purpose
`dev`	Push to `develop`	Active development testing
`staging`	Manual promote / tag	Pre-production validation
`prod`	Merge to `main` + manual approval	Live environment

---

15. Troubleshooting Guide

15.1 Application / Docker Issues

Symptom	Likely Cause	Solution
Container exits immediately	Missing env var or bad `CMD`	Check `docker logs <container>`; verify `.env`
`ImagePullBackOff` in K8s	Wrong image tag or ECR auth expired	Verify tag exists in ECR; re-check IRSA/pull secret
Build fails on `pip install`	Version conflict or network issue in runner	Pin dependency versions; check `requirements.txt`
Health check fails post-deploy	DB/Redis not reachable from Pod	Check Security Group rules and `DATABASE_URL`

15.2 GitHub Actions Issues

Symptom	Likely Cause	Solution
Workflow doesn't trigger	Path filters or branch trigger misconfigured	Review `on:` block in workflow YAML
ECR push `403 Forbidden`	OIDC role trust policy or permissions wrong	Verify IAM role trust relationship with GitHub OIDC provider
SonarQube step times out	Token expired or server unreachable	Rotate `SONAR_TOKEN` secret; check server URL/network
Trivy fails the build unexpectedly	New CVE published in base image	Bump base image version or add a documented ignore with justification

15.3 Terraform Issues

Symptom	Likely Cause	Solution
`Error: state lock`	Previous apply crashed, DynamoDB lock stuck	`terraform force-unlock <lock-id>` (after confirming no concurrent run)
Resource already exists error	Manual change made outside Terraform	`terraform import` the resource or remove the manual change
Plan shows unwanted destroy/recreate	Immutable attribute changed (e.g., subnet CIDR)	Review change carefully; may require phased migration

15.4 Kubernetes Issues

Symptom	Likely Cause	Solution
`CrashLoopBackOff`	App crashing on startup (bad config/secret)	`kubectl logs <pod> --previous`; check ConfigMap/Secret values
`Pending` Pods	Insufficient node capacity	Check Cluster Autoscaler logs; verify node group max size
Ingress has no address	Load Balancer Controller not installed/misconfigured	`kubectl describe ingress`; check controller Pod logs
`readiness probe failed`	App slow to start or dependency unavailable	Increase `initialDelaySeconds`; verify DB/Redis connectivity

15.5 Networking / AWS Issues

Symptom	Likely Cause	Solution
Domain doesn't resolve	Route 53 record missing/misconfigured	Verify hosted zone NS records match domain registrar

Symptom	Likely Cause	Solution
ACM certificate stuck “Pending Validation”	DNS validation CNAME not created	Add the CNAME record Terraform/console provides
WAF blocking legitimate traffic	Overly aggressive managed rule	Review WAF sampled requests; add exception rule
High latency from certain regions	CloudFront cache misses / no edge caching for API	Confirm caching behavior is only applied to static content, not dynamic API paths

16. Best Practices

16.1 Enterprise / General

- Treat infrastructure, pipelines, and manifests with the same code-review rigor as application code.
- Keep environments (`dev` / `staging` / `prod`) as close to identical as possible (“dev/prod parity”).
- Document every deliberate trade-off (see Section 1.4) so future readers understand *why*, not just *what*.

16.2 Security

- Never disable a failing security gate to “unblock” a merge — fix the root cause or file a documented, time-boxed exception.
- Rotate credentials in Secrets Manager on a schedule, not only when compromised.
- Apply the principle of least privilege to every IAM role, Security Group, and Kubernetes RBAC binding.

16.3 Cloud

- Tag every AWS resource (`Project` , `Environment` , `Owner`) for cost tracking and cleanup.
- Destroy `dev` /test environments when not actively in use to control cost.
- Prefer managed services (RDS, ElastiCache, EKS) over self-managed equivalents for a student team’s operational bandwidth.

16.4 Git

- Keep PRs small and focused — easier to review, easier to revert.
- Never force-push to `main` or `develop` .
- Squash-merge feature branches to keep history readable.

16.5 Python

- Fail fast with clear validation errors rather than silent failures.
- Keep business logic out of route handlers — route handlers should be thin.
- Write tests before considering a feature “done,” not after.

16.6 Kubernetes

- Always set resource `requests` / `limits` — never leave them unbounded.
- Use readiness probes to prevent traffic from reaching a not-yet-ready Pod.
- Avoid `latest` tags in any manifest; always deploy explicit, immutable tags.

---

17. Final Appendix

17.1 Repository Folder Structure (Full)

```
ecommerce-platform/
├── services/
│   ├── product-service/
│   ├── cart-service/
│   ├── inventory-service/
│   ├── order-service/
│   └── notification-service/
├── infrastructure/
│   └── terraform/
│       ├── modules/
│       └── environments/
├── k8s/
│   ├── base/
│   └── overlays/
├── .github/
│   └── workflows/
│       ├── ci.yml
│       ├── terraform-plan.yml
│       ├── terraform-apply.yml
│       └── cd-deploy.yml
├── docs/
│   └── DevSecOps-ECommerce-Handbook.md
└── README.md
```

17.2 Architecture Diagram (Consolidated)

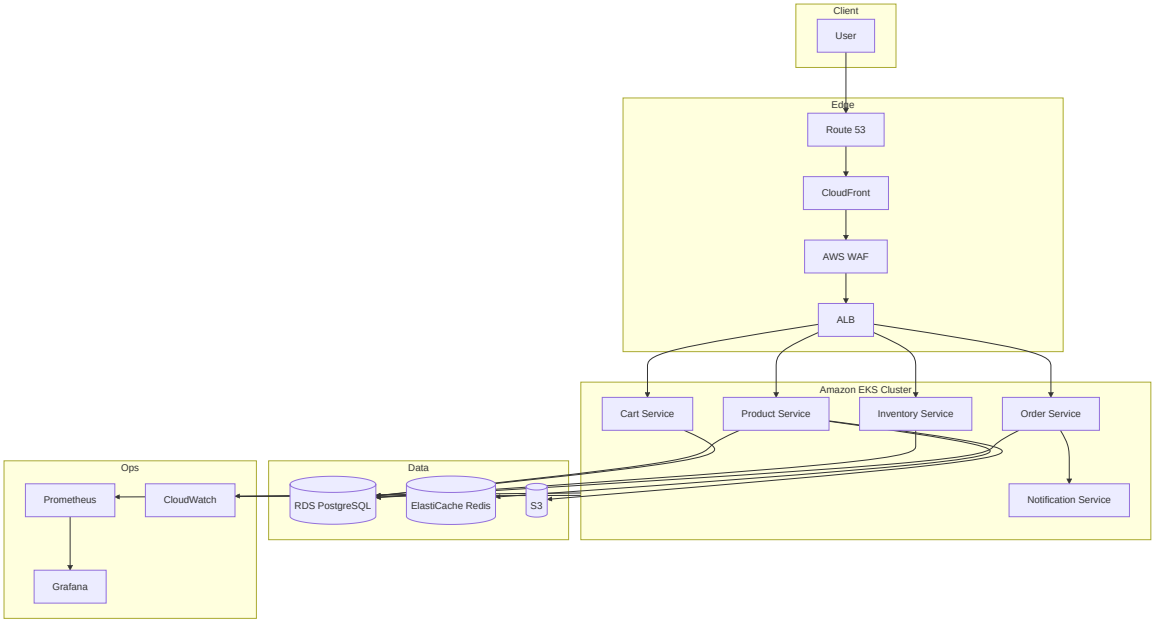


Diagram 12

17.3 Useful Commands Cheat Sheet

AWS CLI

```
aws sts get-caller-identity
aws eks update-kubeconfig --name ecom-dev-eks --region us-east-1
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin <account_id>.dkr.ecr.us-east-1.amazonaws.com
aws secretsmanager get-secret-value --secret-id ecom/prod/db-credentials
```

kubectl

```
kubectl get pods -n ecom-prod
kubectl logs <pod-name> -n ecom-prod --previous
kubectl describe pod <pod-name> -n ecom-prod
kubectl rollout status deployment/product-service -n ecom-prod
kubectl rollout undo deployment/product-service -n ecom-prod
kubectl port-forward svc/product-service 5000:80 -n ecom-prod
```

Terraform

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
terraform destroy
terraform force-unlock <lock-id>
```

Docker

```
docker build -t product-service:local .
docker run -p 5000:5000 --env-file .env product-service:local
docker-compose up --build
docker system prune -a
```

17.4 GitHub Actions Workflow Summary

Workflow File	Trigger	Purpose
`ci.yml`	PR to `develop` / `main`	Lint, test, SAST/SCA/secret/image scans
`terraform-plan.yml`	PR touching `infrastructure/**`	Post `terraform plan` output as PR comment
`terraform-apply.yml`	Merge to `main` (manual approval)	Apply infrastructure changes
`cd-deploy.yml`	Merge to `develop` / `main`	Build, push image, `kubectl apply`, smoke test

17.5 Glossary

Term	Definition
CI/CD	Continuous Integration / Continuous Deployment — automated build, test, and release pipeline
DevSecOps	Practice of embedding security practices within the DevOps pipeline
IaC	Infrastructure as Code — managing infrastructure through machine-readable definition files
SAST	Static Application Security Testing — analyzing source code without executing it
SCA	Software Composition Analysis — scanning third-party dependencies for known vulnerabilities

Term	Definition
EKS	Elastic Kubernetes Service — AWS's managed Kubernetes offering
ECR	Elastic Container Registry — AWS's private Docker image registry
ALB	Application Load Balancer — Layer 7 AWS load balancer
WAF	Web Application Firewall
HPA	Horizontal Pod Autoscaler
IRSA	IAM Roles for Service Accounts — lets Kubernetes Pods assume scoped AWS IAM roles
RTO/RPO	Recovery Time/Point Objective — disaster recovery targets

---

End of Handbook — Enterprise DevSecOps E-Commerce Platform v1.0